PUBLIC

Code Assessment

of the NFAT Facility Smart Contracts

October 13, 2026

Produced for



Sky

by



CHAINSECURITY

Contents

1 Executive Summary 3
2 Assessment Overview 5
3 Limitations and use of report 9
4 Terminology 10
5 Open Findings 11
6 Resolved Findings 12
7 Informational 13
8 Notes 14

1 Executive Summary

Dear all,

Thank you for trusting us to help Sky with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of NFAT Facility according to Scope to support you in forming an opinion on their security risks.

Sky implements an NFAT (Non-Fungible Allocation Token) Facility for bespoke capital deployment deals between Primes and Halos. Primes can queue gem assets, and operators facilitate issuance by forwarding deposits and minting an ERC-721 representing the deal. Repayment and collection complete the lifecycle.

In Version 4, scripts have been added to facilitate `NFATFacility` 's offchain permissionless deployment and onchain Star's governance privileged initialization.

The subjects covered in our audit are asset solvency, functional correctness, and access control. Security regarding all the aforementioned subjects is high.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The following files are in scope of this review:

```
src/NFATFacility.sol
deploy/
    NFATDeploy.sol
    NFATInit.sol
script/DeployNFAT.s.sol
```

`src/NFATFacilityFactory.sol` was added to the scope in Version 3 but was removed in Version 4. `script/DeployNFAT.s.sol` was added to scope in Version 4.

The assessment was performed on the source code files inside the NFAT Facility repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	04 Mar 2026	b9eaaec877f4670ba4b14e0d8858235e2258a1ca	Initial Version
2	17 Mar 2026	530a5e328f6a51895f58bbff86ac981be777ea27	After Intermediate Report
3	08 Jul 2026	97307d4542c66b9f162f2e1a5f9093f17aac083b	Facility Factory Added
4	28 Aug 2026	05ca58db70f0c3660791c86973a8bc085344a513	Deployment scripts
5	13 Sep 2026	b777ff26ebc96aba52fb704195f3a283a02235fd	Script Adjustments
6	13 Oct 2026	fa089fe0507fdc2c369562ae0d6ee262fb128a0e	Init Script Gem Config

For the solidity smart contracts, the compiler version `0.8.24` was chosen and the evm version is set to `cancun`.

2.1.1 Excluded from scope

Any other files are excluded from the scope.

2.2 System Overview

This system overview describes the initially received version (Version 1) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Sky offers an NFAT (Non-Fungible Allocation Token) Facility, which defines a system for bespoke capital deployment deals between Primes and Halos. Primes can queue gem assets in an NFAT Facility, and Halos can claim the deposits and mint an NFAT representing the ownership of the deal. The actual deal terms are tracked offchain.

Each NFAT Facility itself is an ERC721 with one gem token for deposit and one recipient that receives deposited funds. `_update()` is overridden and the NFT transfer is subjected to whitelisting if `identityNetwork` is configured.

NFAT Facility follows Sky's typical access control mechanism. Three privileged roles exist in the system:

1. `wards`: The ultimate admins; responsible for role management, configurations, and funds rescue.
2. `buds`: The operators; responsible for handling the deposit and NFAT minting.
3. `cops`: The freezers; responsible for emergency pause that stops `subscribe()`, `issue()`, `repay()` and `collect()`.

The capital deployment and redemption flow is as follows:

1. `subscribe()` (stoppable): The depositor can queue a deposit.
2. `issue()` (stoppable): The bud can forward the deposit to the recipient and mint an NFAT to the depositor representing a claim on the capital deployment. Note one deposit could result in issuance of multiple NFATs.
3. `withdraw()`: The depositor can withdraw unfilled deposit at any time.
4. `repay()` (stoppable): Anyone can permissionlessly repay the deployed capital to an existing NFAT.
5. `collect()` (stoppable): The NFAT owner can claim the repaid funds. Note the `identityNetwork` whitelisting applies if configured.

After deployment, the wards need to set the configurations with:

1. `file(bytes32, address)`: Set the recipient address.
2. `file(bytes32, string)`: Set the `baseURI`.

Further the wards can rescue the funds in different stages:

1. `rescue()`: A general rescue function that can directly transfer any token with designated `to` address and amount.
2. `rescueDeposit()`: It decreases the queued deposit of a user and transfers the amount to a designated `to` address.
3. `rescueCollectable()`: It decreases the `collectable` of a user and transfers the amount to a designated `to` address.

2.2.1 Deployment Scripts

`NFATDeploy` deploys a new `NFATFacility` with `SUSDS` as the gem and transfers ownership to the intended owner, expected to be governance.

`NFATInit` validates that the gem matches `SUSDS` and that the ERC-721 name and symbol match the expected configuration. It then sets the `recipient`, `identityNetwork`, and `baseURI` (since Version 2), whitelists the `operator`, registers the `freezers`, and populates the chainlog. Note that governance is expected to fully validate the deployed facility before running the init script (e.g., verifying that no unintended admins exist and similar properties).

2.2.2 Changes in Version 3

In Version 3, the `NFATFacilityFactory` was added. It exposes a single permissionless `deploy()` function that deploys and fully configures a new `NFATFacility` in a single transaction:

1. It deploys a new `NFATFacility` with the provided `gem`, `name`, and `symbol`.
2. It grants the supplied `wards`, `buds`, and `cops` on the new facility. At least one ward must be provided, and every role address must be non-zero.
3. It files the `recipient` (required, non-zero) and, if provided, the `baseURI` and `identityNetwork`.
4. It renounces its own ward role on the newly deployed facility.

The factory emits a `FacilityDeployed` event capturing the full configuration. Anyone can call `deploy()`; each deployed facility is independent, and its administration is fully controlled by the `wards` specified by the caller. The factory retains no privileges over the facilities it creates.

2.2.3 Changes in Version 4

In Version 4, scripts have been added to facilitate `NFATFacility` 's offchain permissionless deployment and onchain Star's governance privileged initialization.

Different from Version 3, the initialization script `init()`:

1. requires that only `NFATFacility` for USDS or SUSDS can be deployed.
2. sets `almProxy` to buds for NFT issuance and leaves `identityNetwork` and `baseURI` as optional configurations.
3. supports multiple operators at initialization now.
4. no longer registers the deployed facility in the Sky chainlog, as the NFAT onboarding spells execute in the context of a Star's SubProxy.

2.2.4 Changes in Version 5

In Version 5, the initialization script `init()`:

1. removes the `operators` array from `NFATConfig`, leaving `cfg.almProxy` as the only bud.
2. rejects re-initialization by requiring that `recipient()` is still unset.

2.2.5 Changes in Version 6

In Version 6, the initialization script `init()`:

1. takes the expected gem address directly in `NFATConfig` (`cfg.gem` replaces the chainlog key `cfg.gemKey`) and no longer depends on `DssInstance`. Consequently, `init()` only checks that `facility.gem()` matches `cfg.gem` and no longer restricts the gem to USDS or SUSDS. This restriction is now only enforced at deployment by `NFATDeploy`. For a facility deployed by other means, governance is expected to validate that `cfg.gem` is the intended token.
2. additionally rejects re-initialization if `identityNetwork()` or `baseURI()` is already set.
3. no longer checks `facility_` against `address(0)`, as the subsequent call to `facility.recipient()` reverts in that case.

2.3 Trust Model

- `wards`: Fully trusted; assumed to be Sky's Star governance. In the worst case, they can drain all deposits and collectables.

- `buds`: Semi-trusted; assumed to be operators to facilitate the capital deployment. In the worst case, they can DoS the NFAT issuance.
- `cops`: Semi-trusted; assumed to be freezers who should pause the contract in emergency. In the worst case, they can temporarily DoS the stoppable operations.
- `recipient`: Semi-trusted; the capital deployment target assumed to repay the funds per offchain terms. In the worst case, all the funds deployed to the recipient could be lost.
- `identityNetwork`: Semi-trusted; assumed to be the Sky Identity Network. In the worst case, it can DoS the NFAT issuance / transfer or allow issuance / transfer to unexpected addresses.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the Resolved Findings section. The findings are split into these different categories:

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the Open Findings section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0
Informational Findings	1

- Unchecked BaseURI at Initialization Code Corrected

6.1 Unchecked BaseURI at Initialization

Informational Version 1 Code Corrected

CS-SKYNFAT-001

The initialization library does not set the BaseURI. Consequently, the `tokenURI()` will return an empty string before BaseURI is set.

Code corrected:

The base URI is now initialized as part of the initialization script. In Version 4, this becomes an optional configuration.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 Factory Sanity Checks

Informational **Version 3** **Code Partially Corrected**

CS-SKYNFAT-002

`NFATFacilityFactory.deploy()` is missing a few input sanity checks. None have a lasting security impact, but they are worth noting for consistency:

1. `gem` is not checked against `address(0)`, unlike every other required address (wards, buds, cops, recipient). As `gem` is immutable with no `file` setter, a zero value cannot be fixed after deployment.
2. Duplicate entries in the `wards` / `buds` / `cops` arrays are applied idempotently on-chain but still emitted verbatim in the `FacilityDeployed` event, so the event can diverge from actual state.
3. The factory's own address (`address(this)`) is accepted in `wards`: it is relied and then unconditionally revoked via `deny(address(this))`. If it is the only entry, the `require(wards.length > 0)` check is defeated and the facility is left with no ward.

Code partially corrected:

The factory was removed and initialization is now done through scripts only. Points 1 and 3 are resolved.

Point 2 still persists in `NFATInit.init()`, which checks `cfg.freezers` against `address(0)` but not for duplicates. Duplicates are applied idempotently on-chain while `addFreezer()` emits on every call. In **Version 5**, the `cfg.operators` array was removed from the script, so this only concerns the freezers.

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Identity Network Change Behavior

Note Version 1

The `identityNetwork` can be changed or added after deployment via `file(bytes32, address)`. Existing NFAT holders may no longer be able to collect repaid funds or transfer their NFATs if they are not recognized by the newly configured identity network.

8.2 NFAT Contains No Term Details Onchain

Note Version 1

Each NFAT simply has a unique ID, and it does not contain any information regarding funds allocated per issuance. Term details live off-chain and can optionally be expressed on-chain via subscribe()'s data parameter and/or encoded in the tokenId if needed.

8.3 One Deposit Can Split Into Multiple NFATs

Note Version 1

`issue()` may consume only part of a deposit for an NFAT issuance. Consequently, one deposit may be split into multiple NFATs. Further, issuance with 0 amount is allowed. In theory, an unlimited amount of NFATs can be issued per deposit.

8.4 Subscription Is Not Whitelisted

Note Version 1

Currently subscription (`subscribe()`) is permissionless and not whitelisted. However, issuance (`issue()`) and redemption (`collect()`) require that the `msg.sender` is whitelisted if `identityNetwork` is configured. Hence a successful subscription does not suggest eligibility of following issuance and collection.

8.5 Unchecked Token Transfer Return Value

Note Version 1

In general, the gem token `transfer()` / `transferFrom()` return values are never checked. In case such a transfer fails without reverting, it creates an inconsistency between:

1. Events and state changes.
2. Accounting variable changes and actual balance change.

8.6 Usage of almProxy in the Initialization Script

Note Version 4

`NFATInit.init()` files `cfg.almProxy` as the `recipient` and additionally whitelists it as a bud via `kiss(cfg.almProxy)`.

The script is intended for the `NFATHaloFacet`, where the ALM proxy is both the recipient and the operator. It is not suitable for the `NFATPrimeFacet`, where the recipient is not necessarily an operator. The naming reflects this intended usage, as the configuration field filed as `recipient` is called `almProxy`.

As of Version 5, the `operators` array was removed from `NFATConfig`, leaving `cfg.almProxy` as the only bud whitelisted by the script. This further strengthens the intended usage, as the script can no longer whitelist an operator distinct from the recipient.